CODEBASE ONBOARDING COMPANION

Requirements Elicitation Report

IT314 — Software Engineering

Compiled from seven stakeholder elicitation activities

September 2026

1. Problem Description

When a company's senior developers have already built a software project, new junior developers who later join the team are handed a large, unfamiliar codebase and expected to become productive quickly. In practice this handover is slow and expensive: new hires do not know where to start, cannot easily locate the files or functions relevant to an assigned task, and end up repeatedly interrupting senior developers for explanations that are never written down. The Engineering Manager loses visibility into how onboarding is actually progressing, and the organisation has no reliable way to shorten the time it takes a new developer to reach their first meaningful contribution.

The Codebase Onboarding Companion is an AI-assisted platform designed to close this gap. A developer connects a GitHub repository, and the system parses its files, folders, modules, functions, and dependencies into an interactive Codebase Map, detects the technology stack, and generates plain-language, source-referenced explanations of how the project works. A conversational chatbot lets the developer ask natural-language questions and receive answers grounded in the actual code, with exact file and line citations.

Three supporting features extend this core experience: Knowledge Concentration, which shows where historical contribution and expertise are concentrated; Code Evolution Timeline, which reconstructs how key modules changed over the project's Git history; and Change Preview, which surfaces the files, dependencies, and tests likely affected before a developer edits a function. Building this system safely and usefully requires input not only from the junior developer who uses it directly, but from every party whose work, infrastructure, or data the system touches — which is why requirements were elicited from seven distinct stakeholders, two of them external services rather than people.

2. Stakeholders Considered and Why

Seven stakeholders were included in the elicitation process. Five are human/organisational roles and two are external services the system depends on but cannot interview directly.

Stakeholder	Why This Stakeholder Was Included
Junior Developer	The primary end user and the person actually blocked by an unfamiliar codebase; the product exists to solve the problem this stakeholder experiences directly, so their friction is the core source of requirements.
Senior Developer / Codebase Owner	Currently absorbs the hidden cost of onboarding and must trust the system enough to let it operate on their code and represent it to new hires; without their trust the tool cannot be safely adopted.
Engineering Manager	Accountable for onboarding outcomes (time-to-first-contribution, support effort) and needs visibility into progress at an aggregate level without exposing individual developers' private activity.
DevOps / Platform / IT Team	Owns the infrastructure, deployment pipeline, and operational stability the platform runs on, and must ensure it scales, recovers from failure, and stays within cost and security bounds.

Stakeholder	Why This Stakeholder Was Included
Security Team	Responsible for ensuring that repository code, credentials, user data, and everything sent to the AI service remain protected from unauthorised access or leakage.
GitHub (external service)	The platform the system integrates with for all repository data (files, history, issues); its documented APIs, authentication model, and rate limits directly constrain the design.
LLM / AI API Provider (external service)	The inference service every explanation, chat answer, and structured extraction depends on; its context limits, streaming behaviour, rate limits, and data-retention policy shape the system's core AI features.

3. Elicitation Technique Used per Stakeholder

Each stakeholder was approached with the technique best suited to how that stakeholder could realistically be reached, as stated in the underlying elicitation reports.

Stakeholder	Elicitation Technique Used	Why This Technique (as stated in the report)
Junior Developer	Structured Questionnaire (Google Form, 29 responses)	A survey lets prospective users report onboarding behaviour, difficulties, information needs, and trust in cited answers in a consistent, comparable format across a broad respondent pool.
Senior Developer / Codebase Owner	Structured Questionnaire / Survey (12 responses)	A form-based survey standardises measurable concerns — time spent answering questions, the safeguards wanted before trusting an AI explainer — so responses can be compared and ranked.
Engineering Manager	Structured Questionnaire / Google Form (11 responses)	"A questionnaire is useful for collecting consistent answers on measurable management concerns." It lets respondents identify onboarding problems, indicate typical onboarding time, select useful capabilities, and state privacy preferences in a structured format.
Security Team	Stakeholder Interview (simulated transcript)	Security concerns need probing, follow-up questions (e.g. what happens to a temporary clone, what happens on an unauthorised access attempt) that a static form cannot capture, so an interview format was used, explicitly labelled as simulated since a real security professional was not available.
DevOps / Platform / IT Team	Three-stage triangulation: Architecture & Data-Flow Analysis, Brainstorming & Operational Checklist Review, and a Comprehensive Stakeholder Interview	"To eliminate operational blind spots, requirements were gathered using a three-stage triangulation process," moving from tracing the system's data flow, to auditing it against an SRE/ORR checklist, to a formal interview that quantified the resulting constraints into concrete thresholds.

Stakeholder	Elicitation Technique Used	Why This Technique (as stated in the report)
GitHub	Document Analysis (official GitHub API and security documentation)	GitHub is an external platform, not a human stakeholder who can be interviewed, so its official REST API, authentication, permissions, and rate-limit documentation was reviewed and converted into requirements instead.
LLM / AI API Provider	Document Analysis (official provider platform documentation)	"Our LLM API provider... is an external cloud service and cannot sit in a meeting," so the standard technique of Document Analysis was applied to the provider's structured-output, streaming, rate-limit, and enterprise-privacy documentation.

4. Quick Report of Elicitation Findings

Condensed across all seven elicitation activities, the following points capture the findings that most directly shaped the requirements in Section 5:

- Junior developers' biggest blockers are not knowing where to start and not being able to locate files/functions relevant to a task, while 93.1% said file-and-line citations would increase their trust in an AI answer (29 respondents).
- Senior developers rated a wrong dependency answer as risky as or riskier than a wrong summary (92%), and named exact file/line citation on every answer as the single strongest safeguard they wanted (83% of 12 respondents).
- Engineering Managers consistently wanted an aggregate onboarding-progress dashboard (100% of 11 respondents) but were equally consistent that individual developers' questions and conversations must stay private from management by default.
- The Security Team interview centred on authenticated, permission-checked repository access, excluding secrets/.env files from AI processing, and deleting temporary repository clones once analysis is complete.
- The DevOps/IT triangulation surfaced concrete operational risks — out-of-memory failures on large monorepos, request-thread blocking during cloning, and GitHub/LLM rate-limit exhaustion — and the corresponding need for asynchronous workers, shallow clones, and throttling.
- Document analysis of GitHub's API confirmed hard constraints the system must design around: least-privilege token scopes, documented rate-limit headers, and defined error/retry behaviour.
- Document analysis of the LLM provider's documentation confirmed that structured JSON-schema output, token-streaming, and an available Zero Data Retention mode are all real capabilities the system should rely on rather than assume.

5. Requirements — Overall System

The requirements below are consolidated across all seven stakeholders into a single specification for the system as a whole, rather than repeated per stakeholder.

5.1 Functional Requirements

ID	Requirement	Description
FR-01	Repository Connection & Analysis	Allow a developer to connect a GitHub repository and branch, then automatically scan its files and folders.
FR-02	Technology Stack Detection	Detect and display the primary languages, frameworks, libraries, and package managers in the repository.
FR-03	Codebase Structure Explorer	Present an organised view of the repository, highlighting key files, modules, and entry points rather than a flat file list.
FR-04	Semantic Code Search	Allow natural-language search over the codebase, returning relevant files/functions beyond exact keyword matches.
FR-05	Codebase Question-Answering	Provide a chatbot that answers free-form questions using the actual repository content, retaining conversational context.
FR-06	Source-Referenced Answers	Attach at least one specific file and line-number citation to every AI-generated answer or explanation.
FR-07	Code Flow Tracing	Trace how a named feature moves across the relevant files/layers of the project in a readable, ordered sequence.
FR-08	Personalised Onboarding Plan	Generate an onboarding plan based on the developer's stated experience, known technologies, and assigned task.
FR-09	Interactive Learning Modules	Break the project into learning modules (e.g. Overview, Frontend, Backend, Auth, Database) tied to real code.
FR-10	Onboarding Progress Tracking	Track and display a developer's completion progress across modules and tasks, persisted across sessions.
FR-11	Setup & Environment Guide	Generate step-by-step setup instructions (dependencies, environment variables, run commands) from the project's own configuration.
FR-12	Setup Troubleshooting	Accept an error message and suggest a likely cause or fix based on the project's configuration.
FR-13	Task-to-Code Guidance	Given a short task description, point the developer to the relevant module, files, and functions to start from.
FR-14	Change Preview / Impact Awareness	Before a file is modified, list the other files, modules, and tests that directly or indirectly depend on it.
FR-15	Knowledge Concentration	Show where historical contribution and code expertise are concentrated, based on commit history.

ID	Requirement	Description
FR-16	Code Evolution Timeline	Reconstruct how key modules changed over the project's Git history.
FR-17	Review & Approval Workflow	Allow a senior developer to review, correct, or approve an AI-generated explanation before it is shown to interns.
FR-18	Question Visibility Dashboards	Give senior developers and managers a dashboard of new-hire questions (senior: individual and flagged low-confidence; manager: aggregate only).
FR-19	Management Reporting	Provide the Engineering Manager with aggregate onboarding progress, outcome metrics, and bottleneck indicators.
FR-20	Access Control & Secret Exclusion	Authenticate users, verify repository permission before analysis, restrict results to authorised users, detect and exclude sensitive files (e.g. .env, keys) from AI processing, and securely delete temporary clones.
FR-21	GitHub API Integration	Retrieve repository content, metadata, and issue information through supported GitHub REST endpoints, honouring granted permissions and documented rate-limit/error behaviour.
FR-22	LLM API Integration	Assemble context-bounded prompts, use structured (schema-enforced) output for graphs/maps, stream responses to the UI, and back off on rate-limit responses.
FR-23	Asynchronous Job Processing	Offload repository cloning, parsing, and embedding generation to background workers that scale independently of the web/API layer.
FR-24	Platform Reliability Operations	Run automated CI/CD with rollback, expose health/metrics endpoints, take scheduled backups, and enforce per-job compute/token cost caps.

5.2 Non-Functional Requirements

ID	Quality Attribute	Description
NFR-01	Usability	Explanations default to plain, non-jargon language; management views are interpretable without reading source code.
NFR-02	Groundedness / Trustworthiness	No explanation is shown without a verifiable source reference; unsupported answers are explicitly flagged rather than fabricated.
NFR-03	Accuracy	Code-flow traces and dependency listings reflect the actual source code rather than an approximation.
NFR-04	Performance	A typical repository scan completes within a few minutes; question-answering and dashboards respond within a few seconds under normal load.

ID	Quality Attribute	Description
NFR-05	Data Privacy & Confidentiality	Repository content is not repurposed beyond the owner's session; proprietary code and answers stay within the owning team's boundary; individual developer queries stay private from management by default.
NFR-06	Security	GitHub permissions follow least privilege; credentials and tokens are never stored or logged in plain text; every backend request enforcing repository or analysis access is authorization-checked.
NFR-07	Reliability	The assistant remains usable for the full onboarding duration without requiring manual intervention; reported metrics stay consistent across repeated views.
NFR-08	Scalability & Elasticity	The system remains usable from small repositories to large monorepos; stateless services and worker pools scale horizontally with load.
NFR-09	Maintainability & Portability	Indexed repository understanding is refreshable on demand; the system is fully containerised with an LLM-provider-agnostic adapter layer.
NFR-10	Auditability	Generated explanations, low-confidence flags, senior-developer corrections, security events, and metric definitions are logged and retrievable.
NFR-11	Availability	Core UI, Codebase Map, and chatbot functionality maintain a high uptime target during standard operating windows.
NFR-12	Fault Tolerance / Resilience	Outbound calls to GitHub and the LLM API use bounded timeouts, circuit breakers, and exponential backoff rather than hanging or crashing.
NFR-13	Observability	Structured logs with correlation IDs and metrics (queue depth, latency, error rate, token counts) are exposed for monitoring and alerting.
NFR-14	Disaster Recovery	Persistent data is restorable from automated backups within a defined Recovery Time and Recovery Point Objective.
NFR-15	Cost Governance	Compute time and LLM token usage are tracked and capped per job and per tenant to keep operating cost predictable.

5.3 Domain Requirements

ID	Requirement
DR-01	Access to any repository-derived view (Codebase Map, Knowledge Concentration, Code Evolution Timeline, Change Preview) shall follow the same permissions granted by the repository hosting platform.
DR-02	Repository and third-party API access shall follow the principle of least privilege throughout the system.
DR-03	Secrets, credentials, and environment files shall not be unnecessarily exposed to AI processing.

ID	Requirement
DR-04	Temporary repository copies and other ephemeral analysis data shall not be retained longer than a defined retention period.
DR-05	Onboarding shall follow a defined lifecycle (repository understanding → learning/setup → task preparation → first meaningful contribution), with "first meaningful contribution" defined consistently by an agreed event such as an accepted commit or pull request.
DR-06	Management-facing reporting shall use aggregate or anonymised data wherever individual developer activity is not required, consistent with organisational privacy policy.
DR-07	Integration with the GitHub and LLM external services shall operate within their documented platform terms, permission models, and rate-limit policies.

6. User Stories

A representative set of user stories is given below, one or two per stakeholder, each as a front-of-card story and a back-of-card set of acceptance criteria.

6.1 Junior Developer

FRONT OF CARD — US-JD-01	BACK OF CARD — Acceptance Criteria
As a junior developer, I want to connect my project's repository and get an automatic overview of its structure and technology stack, so that I don't have to manually explore hundreds of files before understanding what the project even contains.	– Given a valid repository URL and branch, connecting it produces a folder/file structure within a reasonable time. – The system correctly lists the primary languages, frameworks, and package managers detected. – Important files and entry points are highlighted, not presented as a flat, unranked file list. – If the repository is invalid or inaccessible, a clear error message is shown instead of a silent failure.

FRONT OF CARD — US-JD-02	BACK OF CARD — Acceptance Criteria
As a junior developer, I want to know what else in the project might be affected before I change a file, so that I don't accidentally break something I didn't know was connected.	– Given a file about to be modified, an impact check lists the files/modules that directly depend on it. – The list distinguishes direct usage from indirect/possible usage where this can be determined. – The feature covers function- and file-level dependencies without overstating certainty about runtime behaviour.

6.2 Senior Developer / Codebase Owner

FRONT OF CARD — US-SD-01	BACK OF CARD — Acceptance Criteria
As a senior developer, I want every AI-generated explanation to show me exactly which file and line it came from, so that I can verify it in seconds instead of re-deriving the answer myself.	– Every answer displays at least one source file reference alongside the explanation text. – Selecting a source reference opens the exact file and line range. – If no source can be cited, the system states it cannot answer confidently instead of showing an uncited answer.

FRONT OF CARD — US-SD-02	BACK OF CARD — Acceptance Criteria
As a senior developer, I want to review and correct AI-generated module or architecture summaries before interns see them, so that the first explanation a new hire reads is one I've already approved.	– Generated summaries have a draft/pending state until a senior developer approves them. – Edits made by a senior developer are saved and used in place of the AI draft going forward. – Interns can see whether a summary is owner-reviewed or AI-only.

6.3 Engineering Manager

FRONT OF CARD — US-EM-01	BACK OF CARD — Acceptance Criteria
As an Engineering Manager, I want to view the overall onboarding progress of new developers, so that I can understand whether their onboarding is moving forward as planned.	– Given a developer has an active onboarding plan, opening the management view displays current overall progress. – Progress is shown at a useful level of detail (modules/tasks completed) rather than only a raw event log. – The manager can distinguish completed, in-progress, and not-yet-started items.

FRONT OF CARD — US-EM-02	BACK OF CARD — Acceptance Criteria
As an Engineering Manager, I want management reporting to respect developer privacy, so that onboarding visibility does not become unnecessary individual surveillance.	– Individual developer query content is hidden from the normal management view. – Aggregate or anonymised information is used wherever individual data is not required. – Any exception is controlled through explicit authorisation and policy.

6.4 Security Team

FRONT OF CARD — US-SEC-01	BACK OF CARD — Acceptance Criteria
As a Security Team member, I want private repository access to require authentication and authorisation, so that only permitted users can analyse company repositories.	– Given a private repository, an unauthenticated user attempting to analyse it is denied access. – Given an authenticated user without repository permission, analysis is denied. – Given an authenticated user with permission, repository analysis can begin.

FRONT OF CARD — US-SEC-02	BACK OF CARD — Acceptance Criteria
As a Security Team member, I want sensitive files to be identified and excluded from AI processing whenever possible, so that credentials and secrets are not unnecessarily exposed.	– The system identifies configured sensitive file types such as .env and private-key files. – Identified sensitive files are excluded from the AI context whenever possible. – The chatbot does not intentionally send excluded secret files to the LLM.

6.5 DevOps / Platform / IT Team

FRONT OF CARD — US-OPS-01	BACK OF CARD — Acceptance Criteria
As an IT Platform Engineer, I want repository ingestion, AST calculation, and vector embeddings offloaded to asynchronous background workers, so that resource-heavy processing does not degrade responsiveness for active UI and chatbot users.	– When a repository is submitted, the API enqueues a background job and immediately returns a job-tracking identifier. – Synchronous UI queries stay responsive even while background workers run under heavy load. – A worker crash does not interrupt or destabilise the active web/API server processes.

FRONT OF CARD — US-OPS-02	BACK OF CARD — Acceptance Criteria
As a Platform Administrator, I want GitHub API interactions centralised through a managed integration with dynamic backoff, so that shared organisation-wide rate limits are never exhausted.	– The system authenticates via a centrally registered GitHub integration using rotating credentials. – The system monitors remaining rate-limit quota and applies throttling and backoff as it drops toward a critical threshold. – Inbound webhook payloads that fail signature validation are rejected rather than processed.

6.6 GitHub (External Service)

FRONT OF CARD — US-GH-01	BACK OF CARD — Acceptance Criteria
As a platform engineer integrating with GitHub, I want repository content and metadata to be retrieved only through supported, permission-compliant API endpoints, so that the integration keeps working reliably as repositories and access levels change.	– Repository files, directory contents, and repository metadata are read only via documented GitHub REST endpoints. – Requests never exceed the permissions granted to the authenticated token or app. – Rate-limit and error responses from GitHub are handled according to documented behaviour rather than causing an uncontrolled failure.

6.7 LLM / AI API Provider (External Service)

FRONT OF CARD — US-LLM-01	BACK OF CARD — Acceptance Criteria
As a platform engineer integrating with the LLM provider, I want structured, schema-enforced responses and rate-limit-aware request handling, so that generated module maps and explanations are reliable and the system never floods the provider with retries.	– Module maps and dependency matrices are requested using schema-enforced output so responses parse without error. – Streaming responses are forwarded to the client as they arrive rather than waiting for the full completion. – When a rate-limit response is received, subsequent requests are queued and retried with backoff rather than repeated immediately.